SOFTWARE TESTING
CCSW 323

# Testing project Report

ADVANCED TASK MANAGER
WEB APP TESTING

PREPARED BY :

Abrar Alharbi-2317129
Elaf Alsharif-2310115
Ghala Alharbi-2310582
Toleen wael -2311776

Khadija janbi-2312308

# Table of Content

# 01 Software Under Test

## Brief Description:

The selected software is a To-Do List web application developed using HTML, CSS, and JavaScript.

It is a task manager application with a simple user authentication system. The app helps users manage their daily tasks efficiently by allowing them to set priorities, add due dates, and use filtering and search features.

## GitHub Repository Link:

https://github.com/11249a040-sandeep/todo-list-app

## Size of the Software:

HTML: ~120 lines
CSS: ~100 lines
JavaScript: ~150 lines
**Total LOC (excluding comments):** 320 lines

## number classes:

The system does not use object-oriented programming; therefore, it contains no classes.

## number of methods per class:

6 JavaScript functions

# 02 Selected Model and Coverage Criteria

## Testing Models:

### 1- Input Space:

In the Advanced Task Manager Web App, the system behavior is highly dependent on user inputs such as email, password, task name, due date, and priority. Therefore, the input space can be modeled by identifying these parameters and selecting representative values from each domain..

### 2- Graph-Based Testing:

it represents the software as a set of nodes and edges, where nodes represent system states or actions, and edges represent transitions between them,In the Advanced Task Manager Web App, the system behavior involves sequences of user actions such as logging in, adding tasks, marking tasks as completed, and logging out. These actions can be modeled as nodes, while the transitions between them form edges.

# 02 Selected Model and Coverage Criteria

## Coverage Criteria:

**1- Input Space:**
Base Choice Coverage (BCC)
Each Choice Coverage (ECC)
multiple base choice Coverage (MBCC)

**BCC**:
is used because it focuses on a base test case representing normal system behavior, and then varies one input characteristic at a time.

**ECC**:
is used because it ensures that each value of every input characteristic is tested at least once.

**MBCC:**
is used to consider multiple base cases representing different realistic scenarios. It increases test coverage by varying inputs around each base case, making it more effective than using a single base case.

# 02 Selected Model and Coverage Criteria

## Coverage Criteria:

**2- Graph-Based Testing:**
Node Coverage (NC)
&Edge Coverage (EC)

**NC**:
is used to ensure that every node in the graph is executed at least once.,ensures that all major functionalities of the system are tested

**EC**:
is used to ensure that every edge (transition) between nodes is executed at least once. Edges represent the flow between system states,ensures that all possible transitions between actions are tested

# 03 Test Cases

## Base Choice Coverage (BCC) — Input Space Model

### BCC Test Cases

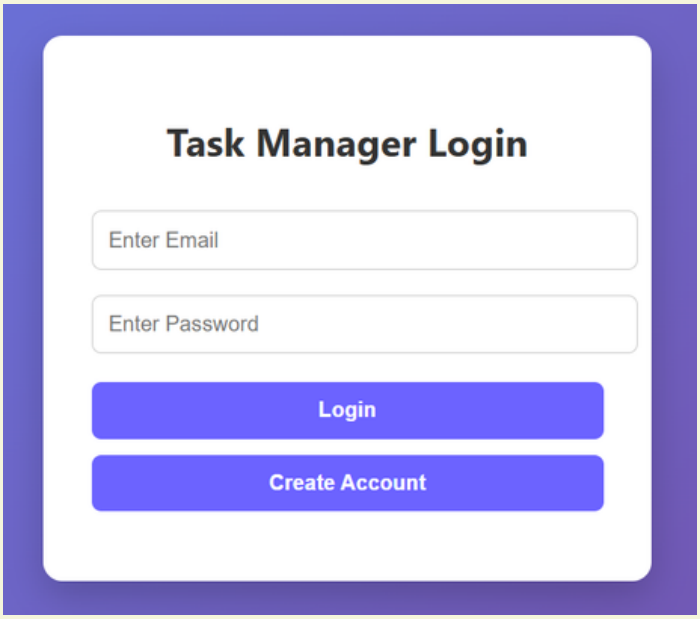
```html
<div class="auth-container">
  <h2>Task Manager Login</h2>

  <input type="email" id="email" placeholder="Enter Email" />
  <input type="password" id="password" placeholder="Enter Password" />

  <button onclick="login()">Login</button>
  <button onclick="register()">Create Account</button>

  <p id="authMessage"></p>
</div>
```

**Input: Task Manager Login**




| | |
|---|---|
| **Characteristics**: Email, Password | **BCC Test Cases:**<br>base={a1 ,b1}<br>A {a2 ,b1}<br>A {a3 ,b1}<br>B {a1 ,b2}<br>B {a1 ,b3} |
| **Blocks:**<br>email: ( valid , empty,wrong)<br>password: ( valid, empty , wrong) | |
| **Abstract IDM:**<br>A = [ a1, a2, a3 ] , B = [ b1, b2,b3 ] | |
| **Number of tests: 1(base)+2+2 = 5** | |

# 03 Test Cases

## Base Choice Coverage (BCC) — Input Space Model

BCC Test Cases Table
email = aa123@gmail.com
password =123456

| Test Case ID | Input Data | Expected Output | Actual Output |
|---|---|---|---|
| BCC-01 | Email: aa123@gmail.com , Password: 123456 | User is redirected to the dashboard and login is successful | login is successful |
| BCC-02 | Email: (empty), Password: 123456 | System shows error message and login is not allowed | “Invalid credentials” message displayed |
| BCC-03 | Email: bb123@gmail.com , Password: 123456 | System shows error message and login is not allowed | “Invalid credentials” message displayed |
| BCC-04 | Email: aa123@gmail.com , Password: (empty) | System shows error message and login is not allowed | “Invalid credentials” message displayed |
| BCC-05 | Email: aa123@gmail.com , Password: 678900 | System shows error message and login is not allowed | “Invalid credentials” message displayed |

# 03 Test Cases

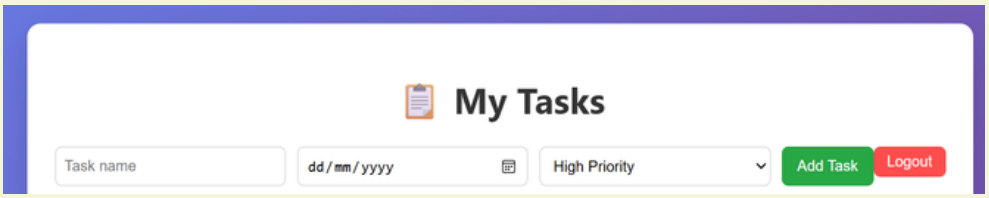
## Each Choice Coverage(ECC) — Input Space Model

### ECC Test Cases

```html
<div class="container">
  <h1>📋 My Tasks</h1>
  <button onclick="logout()" class="logout-btn">Logout</button>

  <div class="task-input">
    <input type="text" id="taskName" placeholder="Task name" />
    <input type="date" id="dueDate" />
    <select id="priority">
      <option value="High">High Priority</option>
      <option value="Medium">Medium Priority</option>
      <option value="Low">Low Priority</option>
    </select>
    <button onclick="addTask()">Add Task</button>
  </div>
```

### Input: add tasks




| | |
|---|---|
| **Characteristics**: name task , due date ,priority | **ECC Test Cases**<br>{a1 , b1, c1}<br>{a2, b2, c2}<br>{a1, b1, c3} |
| **Blocks**:<br>name taske : ( valid , empty)<br>due date : ( valid, empty )<br>priority:(high,medium ,low) | |
| **Abstract IDM:**<br>A = [ a1, a2] , B = [ b1, b2] ,<br>C = [c1,c2,c3] | |
| • **Number of tests: MAX(2,2,3) =3** | |

# 03 Test Cases

## Each Choice Coverage(ECC) — Input Space Model

### ECC Test Cases Table

| Test Case ID | Input Data | Expected Output | Actual Output |
|---|---|---|---|
| ECC-01 | Task: "study ch1", Date: 20/04/2026, Priority: High | Task is added successfully | Task added successfully |
| ECC-02 | Task: "", Date: (empty)Priority: medium | Task NOT added, validation message shown | Validation message displayed |
| ECC-03 | Task: "Gym", Date: 22/04/2026, Priority: Low | Task added successfully with selected priority | Task added successfully |

**Note**:
The number of test cases is 3 because **Each Choice Coverage (ECC)** requires selecting at least one value from each block for every characteristic in at least one test case.

Therefore, 3 test cases were sufficient to cover all input characteristics and their corresponding blocks.

# 03 Test Cases

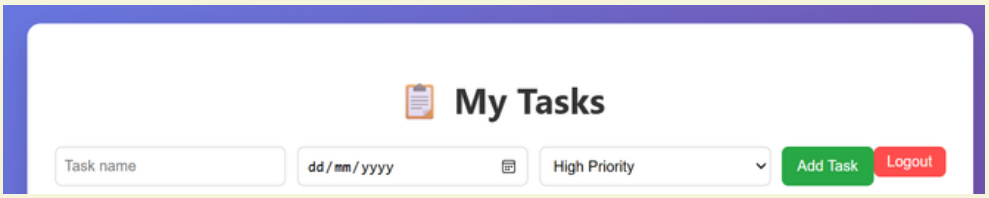
## multiple base choiceCoverage(MBCC) — Input Space Model

### MBCC Test Cases

```html
<div class="container">
  <h1>📋 My Tasks</h1>
  <button onclick="logout()" class="logout-btn">Logout</button>

  <div class="task-input">
    <input type="text" id="taskName" placeholder="Task name" />
    <input type="date" id="dueDate" />
    <select id="priority">
      <option value="High">High Priority</option>
      <option value="Medium">Medium Priority</option>
      <option value="Low">Low Priority</option>
    </select>
    <button onclick="addTask()">Add Task</button>
  </div>
```

### Input: add tasks




| | |
|---|---|
| **Characteristics**: name task , due date ,priority | **MBCC Test Cases**<br>**BASE1** {a1 , b1, c1}<br>**BASE2** {a2, b2, c2}<br>**Tests from a1, b1,c1:**<br>a1, b1, c3,<br>**Tests from a2,b2,c2:**<br>a2, b2, c3, |
| **Blocks**:<br>name taske : ( valid , empty)<br>due date : ( valid, empty )<br>priority:(high,medium ,low) | |
| **Abstract IDM:**<br>A = [ a1, a2] , B = [ b1, b2] , C = [c1,c2,c3] | |

# 03 Test Cases

## multiple base choice Coverage(MBCC)—Input Space Model

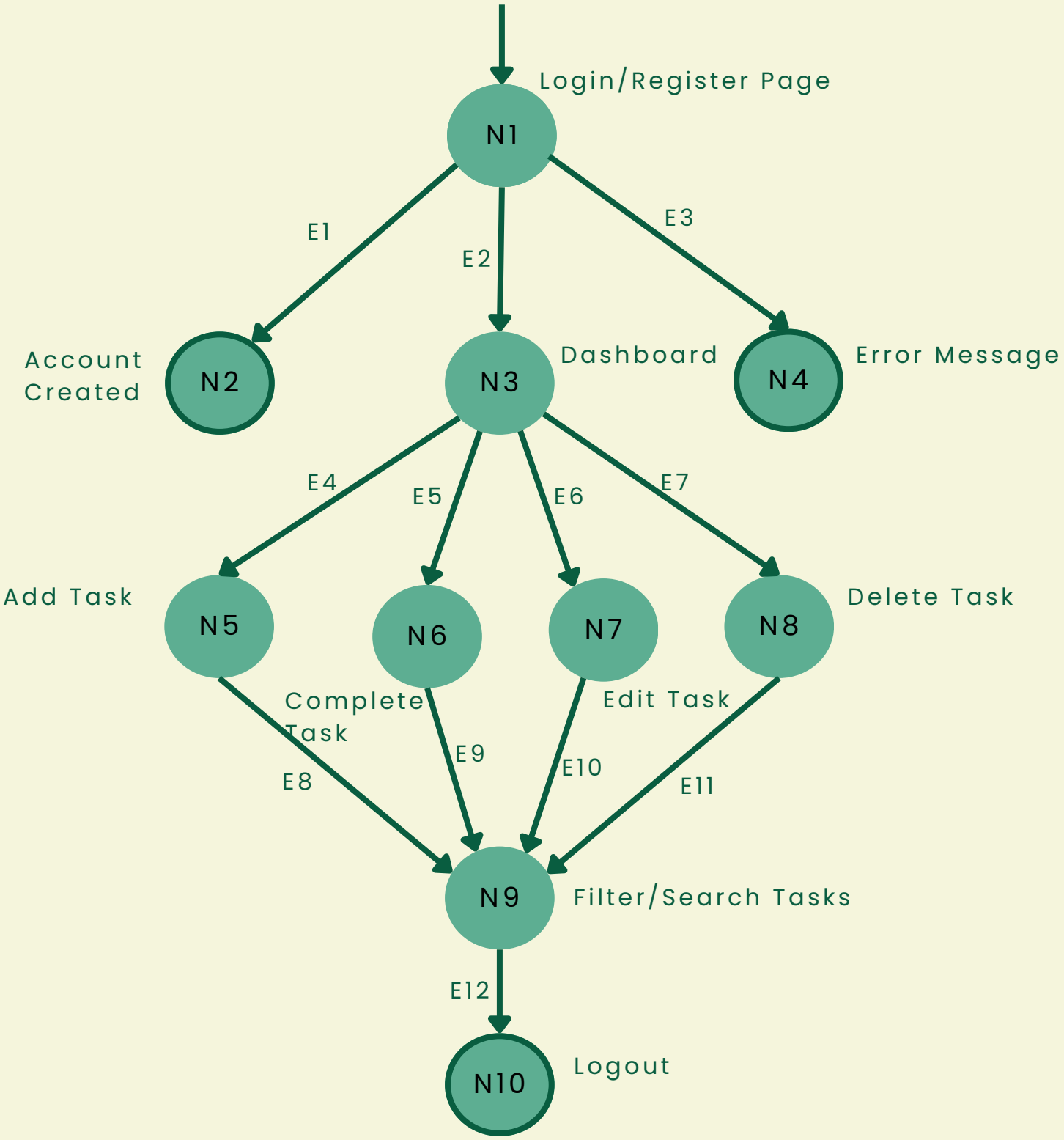
### MBCC Test Cases Table

| Test Case ID | Input Data | Expected Output | Actual Output |
|---|---|---|---|
| MBCC-01 | Task: "study ch1", Date: 20/04/2026, Priority: High | Task added successfully with high priority | Task added successfully |
| MBCC-02 | Task: "", Date: (empty) Priority: medium | Task NOT added, validation message shown | Validation message displayed |
| MBCC-03 | Task: "Gym", Date: 22/04/2026, Priority: Low | Task added successfully with low priority | Task added successfully |
| MBCC-04 | Task: "", Date: (empty) Priority: Low | Task NOT added, validation message shown | Validation message displayed |

03

# Test Cases
## Graph-Based Model

**The following graph represents the system model used to derive node and edge coverage test requirements.**



03

# Test Cases
## Test Requirement

### Node Coverage (NC) Test Requirements

TR(NC) = {N1, N2, N3, N4, N5, N6, N7, N8, N9, N10}

**Node Coverage (NC)** requires that every reachable node in the graph is visited at least once by the test cases. Therefore, the test requirements for **NC** include all nodes representing the system states and actions in the Task Manager application.

### Edge Coverage (EC) Test Requirements

TR(EC) = {
(N1,N2), (N1,N3), (N1,N4),
(N3,N5), (N3,N6), (N3,N7), (N3,N8),
(N5,N9), (N6,N9), (N7,N9), (N8,N9),
(N9,N10)}

**Edge Coverage (EC)** requires that every reachable edge (transition) in the graph is traversed at least once by the test cases. Therefore, the test requirements for **EC** include all transitions between the system states in the Task Manager application.

### Edge Definitions

E1=(N1,N2), E2=(N1,N3), E3=(N1,N4), E4=(N3,N5),
E5=(N3,N6), E6=(N3,N7), E7=(N3,N8), E8=(N5,N9),
E9=(N6,N9), E10=(N7,N9), E11=(N8,N9), E12=(N9,N10)

# 03 Test Cases

## Node Coverage (NC) — Graph-Based Testing

### NC Test Cases Table

| Test Case ID | Input Data | Node Covered | Expected Output | Actual Output |
|---|---|---|---|---|
| NC-01 | Enter a new valid email and password, then click Create Account | N1, N2 | Account is created successfully | Account created successfully |
| NC-02 | Enter valid email and password, then click Login | N1, N3 | User is redirected to dashboard | Login successful |
| NC-03 | Enter invalid email or wrong password, then click Login | N1, N4 | Error message is displayed | “Invalid credentials” message displayed |
| NC-04 | After login, add task “Study testing” with due date and priority | N3, N5 | Task appears in the task list | Task added successfully |
| NC-05 | Mark an existing task as completed | N3, N6 | Task status changes to completed | Task marked as completed |

To be continued...

# 03 Test Cases

## Node Coverage (NC) — Graph-Based Testing

### NC Test Cases Table

| Test Case ID | Input Data | Node Covered | Expected Output | Actual Output |
|---|---|---|---|---|
| NC-06 | Edit task title to “Study software testing” | N3, N7 | Task title is updated in the list | Task updated correctly |
| NC-07 | Delete an existing task | N3, N8 | Task is removed from the task list | Task deleted successfully |
| NC-08 | Use filter/search on existing tasks | N3, N9 | Matching tasks are displayed correctly | Tasks filtered/searched correctly |
| NC-09 | Click Logout | N9, N10 | User is logged out and returned to login page | Logout successful |

# 03 Test Cases

## Edge Coverage (EC) — Graph-Based Testing

### EC Test Cases Table

| Test Case ID | Input Data | Edge Covered | Expected Output | Actual Output |
|---|---|---|---|---|
| EC-01 | Enter a new valid email and password, then click Create Account | E1 | Account created message shown | Account created successfully |
| EC-02 | Enter valid email and password, then click Login | E2 | User is redirected to dashboard | Login successful |
| EC-03 | Enter invalid email or wrong password, then click Login | E3 | Error message shown | “Invalid credentials” message displayed |
| EC-04 | After login, add a task, then use filter/search, then logout | E4, E8, E12 | Task is added, filtered/searched, then user logs out | Task added, filtered/searched, logout successful |

To be continued...

# 03 Test Cases

## Edge Coverage (EC) — Graph-Based Testing

### EC Test Cases Table

| Test Case ID | Input Data | Edge Covered | Expected Output | Actual Output |
|---|---|---|---|---|
| EC-05 | After login, complete a task, then use filter/search, then logout | E5, E9, E12 | Task is completed, filtered/searched, then user logs out | Task completed, filtered/searched, logout successful |
| EC-06 | After login, edit a task, then use filter/search, then logout | E6, E10, E12 | Task is edited, filtered/searched, then user logs out | Task updated, filtered/searched, logout successful |
| EC-07 | After login, delete a task, then use filter/search, then logout | E7, E11, E12 | Task is deleted, filtered/searched, then user logs out | Task deleted, filtered/searched, logout successful |

# 04 Introduction to the Tool

## Selected Tool: Selenium& JUnit

Selenium WebDriver , JUnit for test execution

### Tool Description:

**Selenium** is an open-source automation testing tool designed for web applications.

It allows testers to simulate user interactions with web browsers such as clicking buttons, entering text, and navigating between pages.

**JUnit** is a unit testing framework used in Java to structure and execute test cases, as well as to verify expected results using assertions

### Key Features of Selenium:

- Supports multiple browsers (Chrome, Edge, Firefox)
- Automates real user interactions (click, type, submit)
- Can be integrated with testing frameworks such as JUnit
- Provides accurate and repeatable test execution
- Supports automation of functional and UI testing

**This tool is suitable for the Advanced Task Manager Web App because the application is browser-based and relies heavily on user interaction.**

# 04 Introduction to the Tool

## Tool Setup and Configuration:

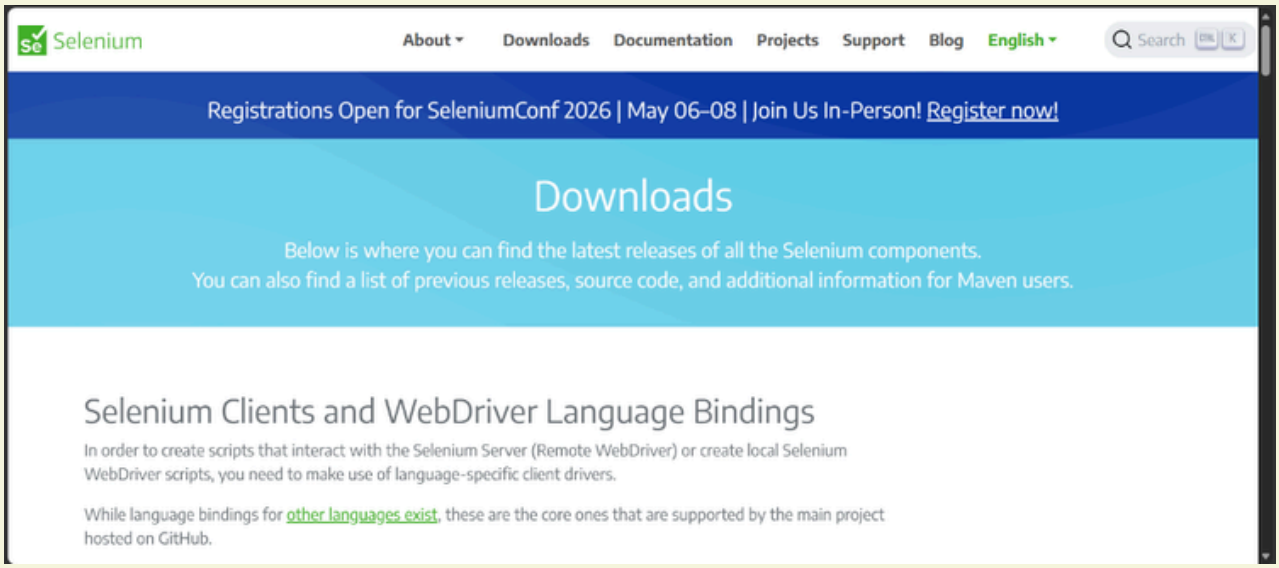
To use Selenium for testing the Advanced Task Manager Web App, the following setup and configuration steps were performed:

**1. Install Java Development Kit (JDK):**
Java was installed to support running Selenium and JUnit test scripts.

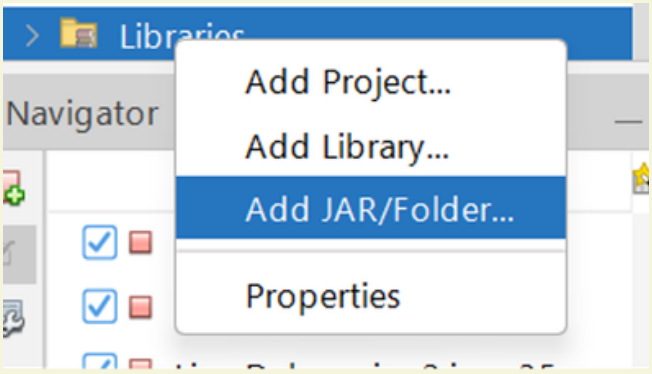
**2. Install an IDE:**
NetBeans (or IntelliJ IDEA) was used to write and execute the test code.

**3. Add Selenium Library:**

Downloading Selenium WebDriver from Official Website



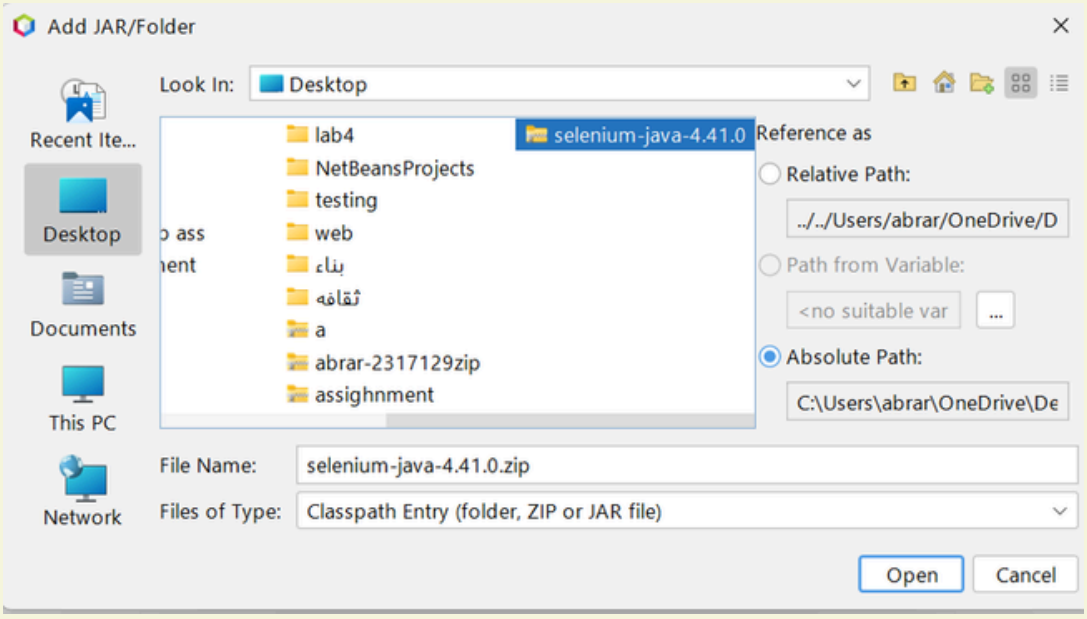

Adding Selenium Library to the Project



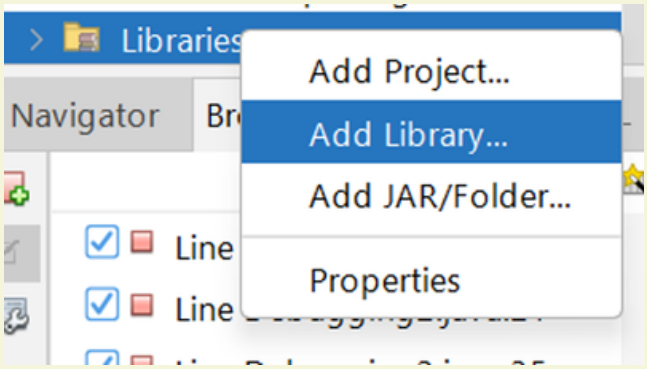
# 04 Introduction to the Tool

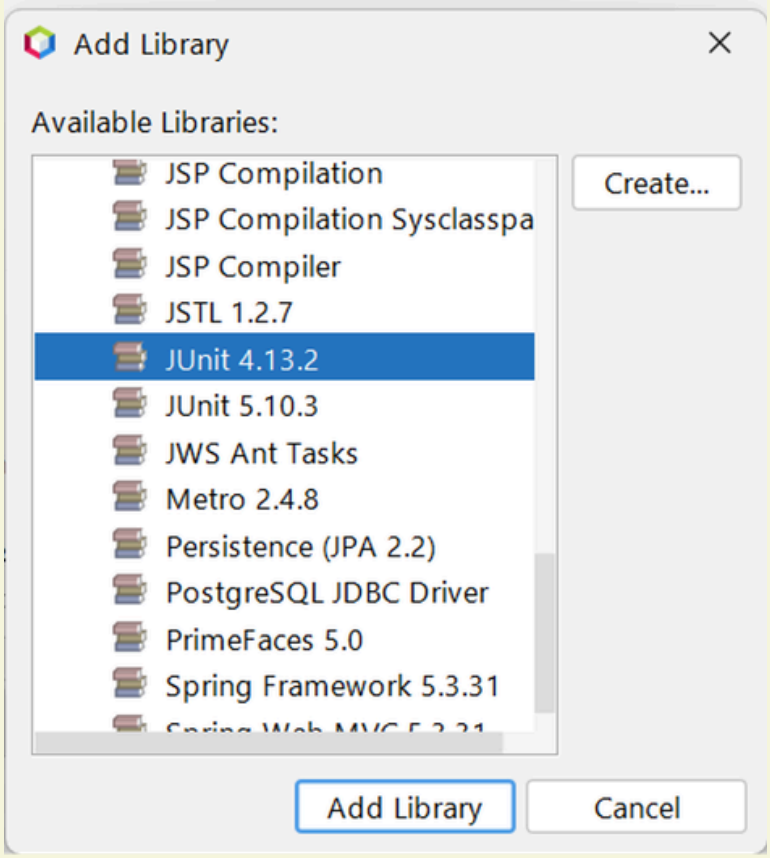
## Tool Setup and Configuration:

### Selecting Selenium WebDriver Files for Integration

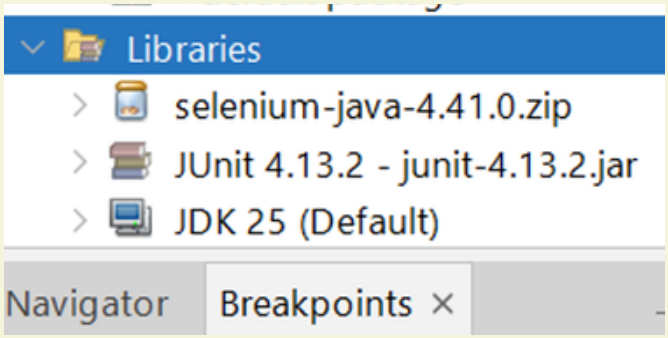



### 4. Add JUnit Library:




**Selenium and JUnit Libraries Successfully Added**

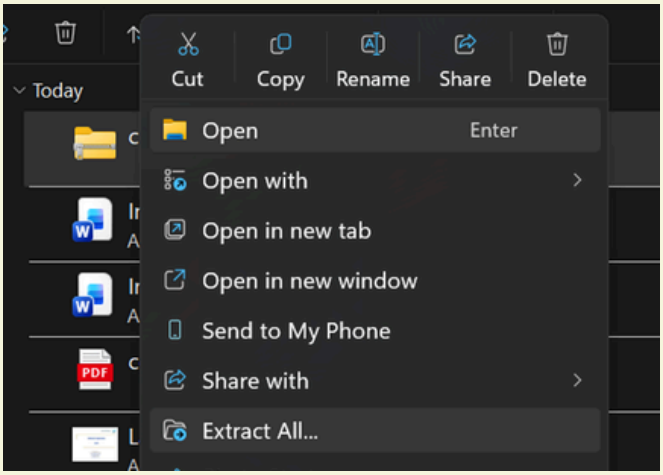


# 04 Introduction to the Tool

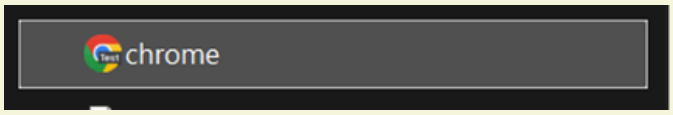
## Tool Setup and Configuration:

### 5. Install WebDriver:

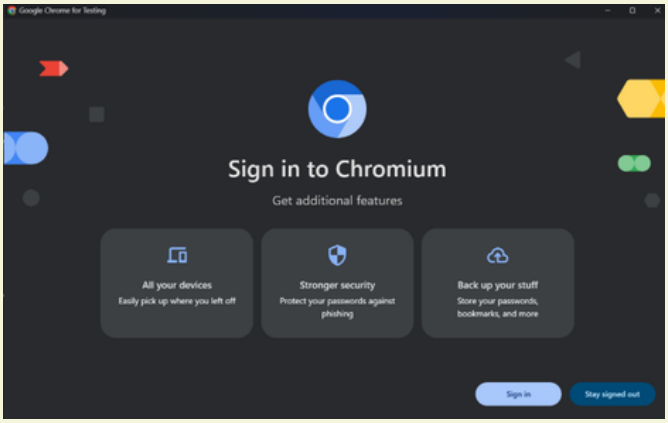
Extracting WebDriver Files

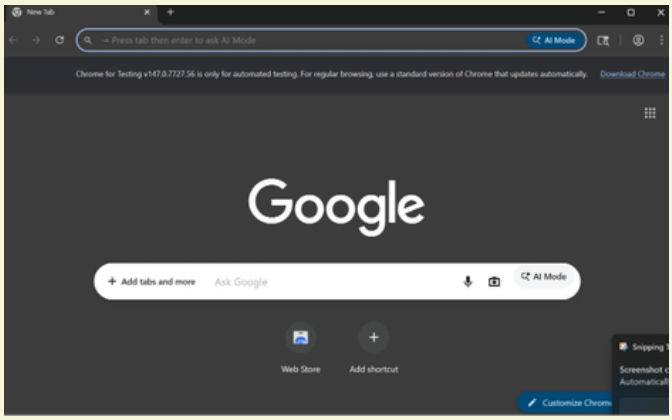



Launching Chrome Browser for Testing




Chrome Browser Ready for Automated Testing



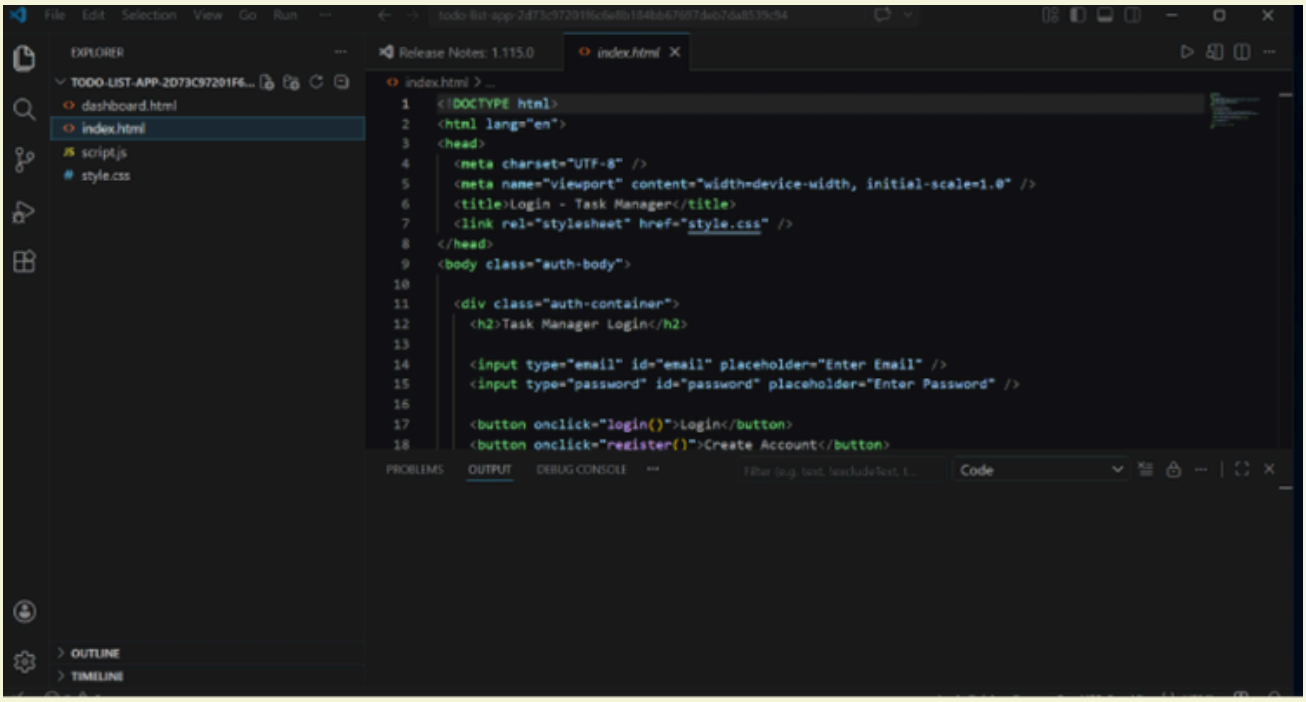

ChromeDriver was installed to enable Selenium WebDriver to control and automate actions in the Chrome browser.

# 04 Introduction to the Tool

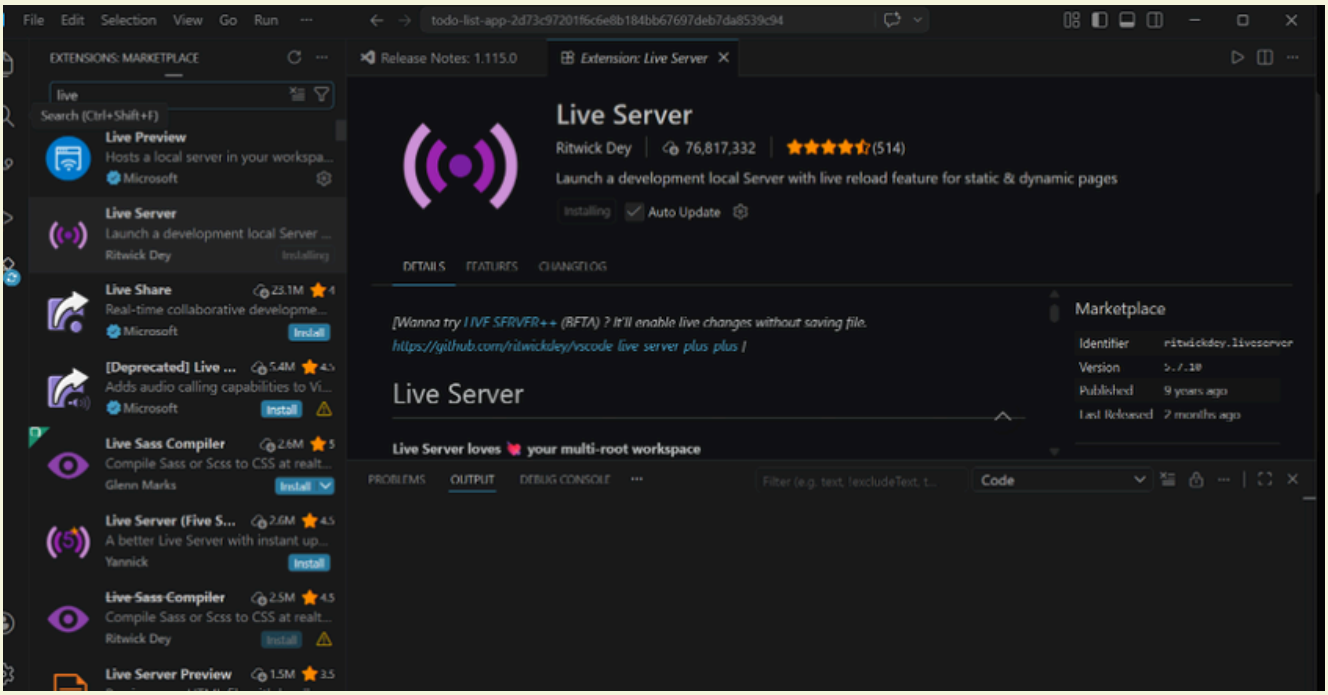
## Tool Setup and Configuration:

### 6. Prepare the Application:

#### Opening the Application in the Development Environment

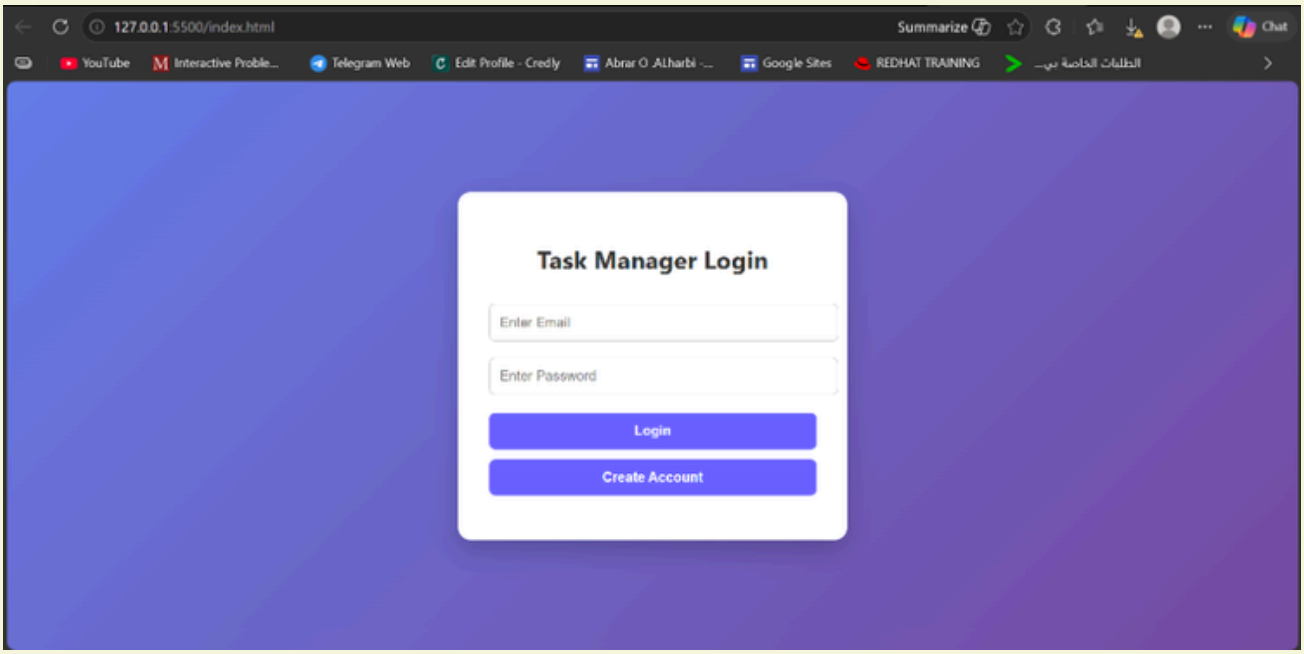



#### Running the Application Using Live Server



## Tool Setup and Configuration:

Running the Web Application on Local Server




The To-do List Web Application was downloaded from GitHub and executed using a local server (Live Server in VS Code) to enable real-time interaction during testing.

### 7. Configure Test Environment:

The correct file path or URL (e.g., http://127.0.0.1/\:5500/index.html) was used in the Selenium script to open the application.

**After completing these steps, the tool was ready to execute automated test cases on the web application.**

# 05 Tool Usage

## Application of the Tool:

**Selenium WebDriver** with **JUnit** was used to execute the designed test cases for the **Advanced Task Manager Web Application.**

First, the application was launched using a local server (Live Server) in Visual Studio Code. Selenium WebDriver was then configured to open the Chrome browser and navigate to the application URL.

The automated test scripts simulated real user actions such as logging into the system, entering task details, selecting a due date, choosing a task priority, and clicking the "Add Task" button.

JUnit was used to run the test cases and verify the results using assertions. The tool compared the expected output with the actual output to determine whether the test case passed or failed.

This automated testing process improved testing accuracy and allowed repeated execution of the test cases efficiently.

# 05 Tool Usage

## Example Test Case Execution Using Selenium and JUnit

**Test Case ID: BCC-02**

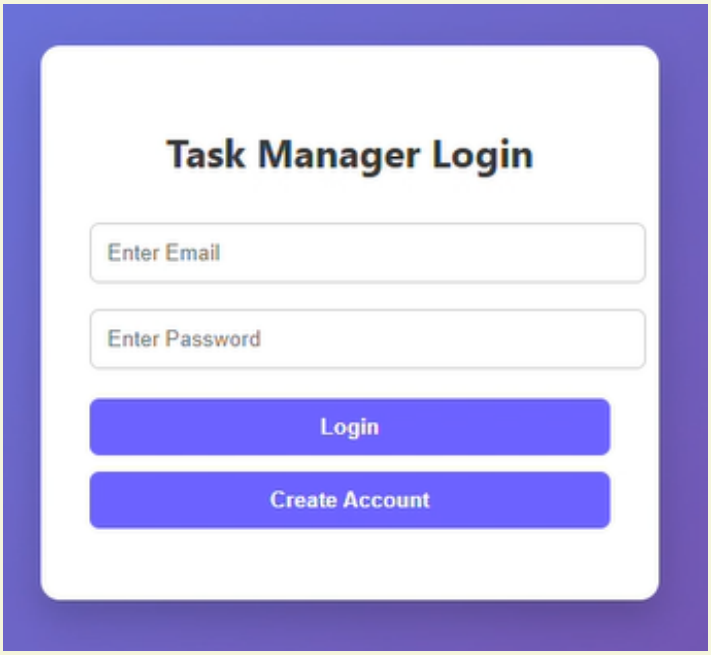
**Description:**
This test case verifies the system behavior when the user attempts to log in with an empty email field.

**Steps**:
1. Selenium WebDriver opened the application in the Chrome browser.
2. The test script left the email field empty.
3. The test script entered a valid password.
4. Selenium clicked the "Login" button automatically.
5. JUnit verified the displayed error message using assertions.

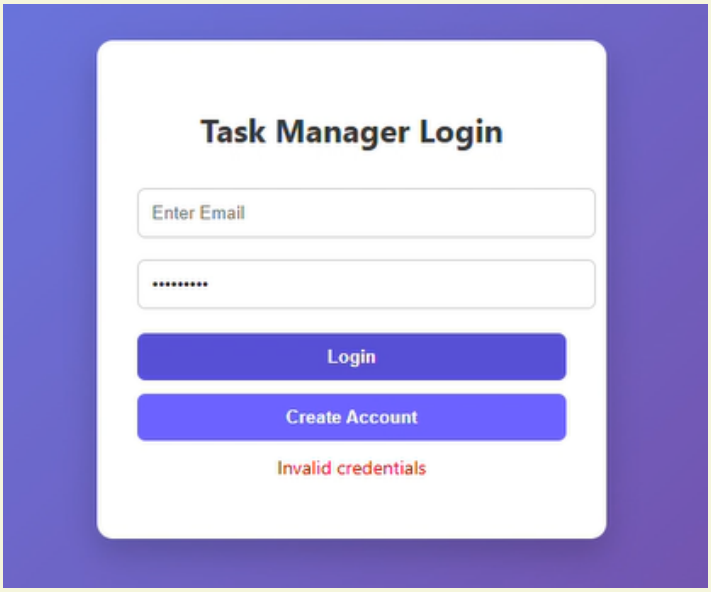
**Expected Result:**
The system should display an error message and prevent login.
**Actual Result:**
The system displayed the message "Invalid credentials" and login was not allowed.
**Status:** Pass

## Test Case Execution Screenshots — BCC-02

### Figure 1: Application opened successfully




### Figure 2: Login attempt with an empty email field resulted in an "Invalid credentials" error message.



## Example Test Case Execution Using Selenium and JUnit

**Test Case ID: ECC-01**

**Description:**
This test case verifies that the system allows the user to add a new task successfully using valid input data.

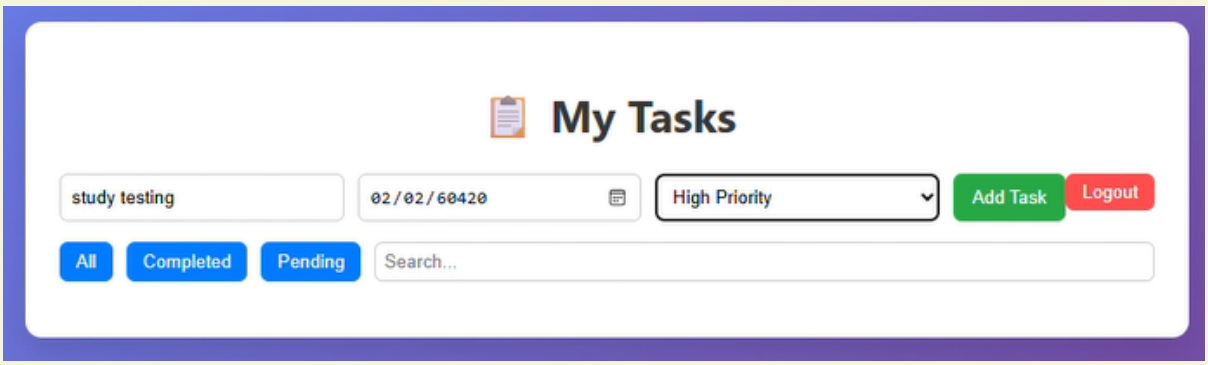
**Steps**:

1. Selenium WebDriver opened the application and logged into the system.
2. The automated test script entered a task name into the task field.
3. The script selected a due date from the date field.
4. The script selected a priority level High from the dropdown list.
5. Selenium clicked the "Add Task" button automatically.
6. JUnit verified that the task was added successfully to the task list.

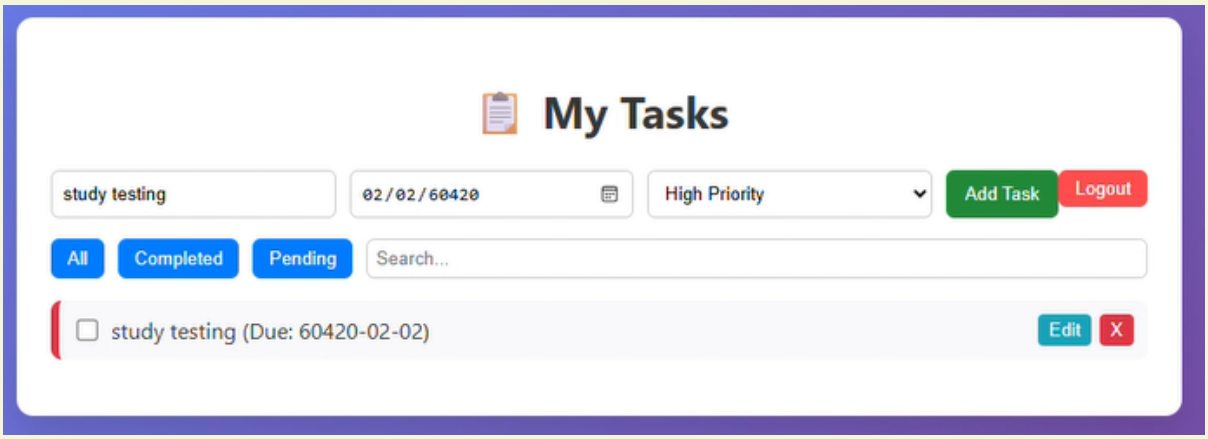
**Expected Result:**
The system should add the new task to the task list successfully.
**Actual Result:**
The task was added successfully and displayed in the task list.
**Status:** Pass

## Test Case Execution Screenshots — ECC-01

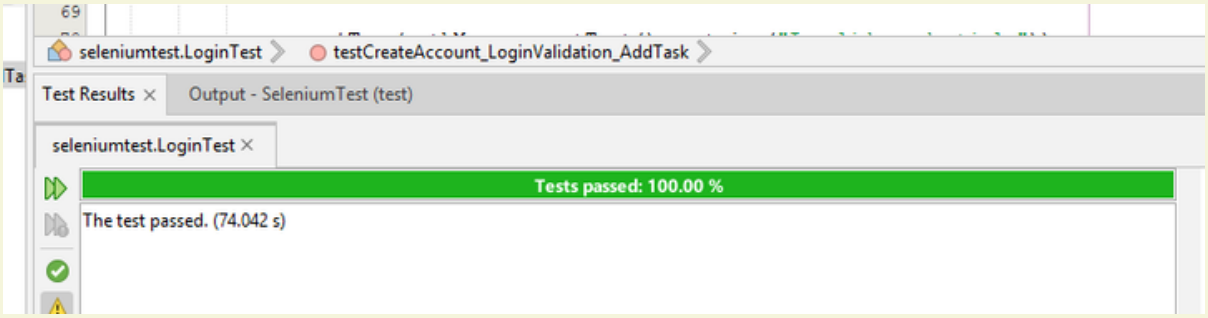
### Figure 3: Task details entered.




### Figure 4: Task added successfully.




### Figure 5: Test result passed for the both exampls.



# 06 Testing Results:

## Number of Tests

**Output of Test Cases:**

A total of 28 test cases were executed using **Selenium WebDriver** and **JUnit** to evaluate the functionality and reliability of the To-Do List Web Application.

**Total Number of Tests:**
28 test cases were executed to verify the system functionality and ensure that all requirements were satisfied.

**Pass/Fail Results:**
All 28 test cases passed successfully, indicating that the system operates correctly according to the defined requirements.

**Number of Passed Test Cases:** 28 (100%)
**Number of Failed Test Cases:** 0 (0%)

**Details of Failures:**
No failures were detected during the execution of the test cases. All test scenarios were completed successfully without errors.

# 07 Conclusion :

The testing process for the Advanced Task Manager Web Application was **successfully completed** using **Selenium WebDriver** and **JUnit**.
The selected testing tool proved to be effective in identifying issues and verifying the correct functionality of the application.

All test cases were executed to evaluate the system's main features, including login validation, task creation, task completion, and task deletion.
The results showed that the application responded correctly to both valid and invalid inputs, and all functionalities performed as expected.

During the testing process, a few challenges were encountered, such as configuring the testing environment and ensuring that the application was properly connected to the local server.
However, these challenges were resolved by correctly installing the required tools and verifying system settings.

**For future improvements**, it is recommended to expand the number of test cases to cover additional scenarios such as search functionality, filtering options, and boundary conditions.
Additionally, integrating automated testing into continuous development processes would further enhance the reliability and quality of the software.

# Plagiarism Checker:

## Plagiarism Checker screenshot: